

JanHelp: AI-Enabled Smart City Complaint Platform

Team JanHelp
Team Lead: Kartik Bhalodiya

Problem and Diagnosis

Urban grievance redressal usually fails before the field team even starts work. Citizens report civic issues with incomplete descriptions, wrong categories, missing location, weak proof, or language barriers. Departments then spend operational time triaging, deduplicating, calling citizens again, and manually deciding who owns the case. JanHelp solves this intake-to-action gap with a Flutter citizen app and a Django backend that converts raw civic pain into a structured, verifiable, routed complaint record.

The repository was reviewed at source level, not only through README files. The implementation contains a Django 5.2 backend at the root, a Flutter mobile client in `smartcity_application/`, 95 HTML templates for web dashboards and emails, 68 Python files across API, models, serializers, AI, email, deployment, and admin logic, and 70 Dart files across screens, providers, services, localization, location, media, and voice/call experiments. Generated folders such as `staticfiles/`, `.dart_tool/`, and build caches exist but were treated as operational artifacts, not product logic.

Product Features and Usefulness

JanHelp is a full civic complaint operating system, not a simple complaint form. It covers citizen access, guided reporting, proof validation, duplicate reduction, department assignment, status tracking, feedback, and reopening. The backend models real city actors: citizens, city admins, departments, department users, managed states and cities, complaint categories, subcategories, dynamic custom fields, complaint media, resolution proof, and reopen proof.

Feature	Usefulness for city and citizen
OTP, password, and guest access	Citizens can report without long onboarding; staff/admin flows still keep authenticated control.
Category, subcategory, and dynamic fields	Complaints become department-ready records instead of vague text blobs.
GPS, manual location, city/state, and map flow	The case can be routed to the right local authority with less back-office correction.
Media and proof workflow	Images and resolution proof create evidence, reduce false reports, and support accountability.
Duplicate detection	Nearby similar complaints can be flagged, reducing repeated work for the same pothole, garbage pile, or water leak.
Department auto-assignment	The model selects an active matching department by city/state scope and distance logic.
Status, SLA timing, rating, and reopen	Citizens see progress; cities get measurable service quality and complaint closure discipline.
Admin dashboards and templates	City teams can monitor load, manage departments/categories, and communicate status by email.
Multilingual AI and call prototype	English, Hindi, Gujarati, chat guidance, and voice-call research improve access for non-technical users.

Technical Architecture and Data Flow

The mobile client is a Flutter application named JanHelp. It uses Provider state management, local secure token storage, route-driven screens, localization, complaint services, category services, location services, upload logic, notifications, and AI/chat services. Its API base is centralized at <https://janhelps.in/api>, so the deployed app can point to the production backend without scattering URLs across screens.

The backend uses Django REST Framework, SimpleJWT authentication, serializers, viewsets, function APIs, Cloudinary signing, email utilities, category APIs, guest tracking, dashboard statistics, and health checks. Deployment support exists through `api/index.py`, `vercel.json`, environment-driven settings, Whitenoise static serving, Cloudinary media storage, and database URL normalization for the Supabase pooler.

The core data flow is: the citizen submits a complaint from the app; Django validates identity or guest fields; serializers normalize language, priority, date, booleans, media, and dynamic field responses; proof images are verified when needed; duplicate checks compare category, subcategory, location radius, and description; the complaint is saved with an **SC** complaint number; the nearest active department is assigned using city/state scope and haversine distance; tracking APIs return current status, SLA timing, media, department, feedback, and reopen information.

AI, Duplicate, and Voice/Call Logic

JanHelp's AI value is practical: it improves data quality before the complaint reaches a department. Backend `complaints/ai_utils.py` uses Gemini with model fallback and image quality checks to reject blank, too-dark, overexposed, unrelated, or weak proof images. `complaints/conversational_ai.py`, `conversation_analyzer.py`, and `step_by_step_ai.py` provide multilingual classification, urgency detection, session state, fallback keyword analysis, guided steps, and category discovery.

The repository also contains advanced voice/call logic that should be presented honestly as a prototype module unless final UI/provider wiring is restored before the live demo. Current Dart files such as `gemini_audio_call_service.dart`, `gemini_live_call_service.dart`, and `call_conversation_manager.dart` implement Gemini Live/native-audio concepts: WebSocket setup, microphone PCM streaming, audio playback, selected language handling, call stage transitions, transcript callbacks, duplicate handling, proof and location stages, and conversion of a voice complaint draft into backend multipart fields. Older documentation describes a language-call screen, while a later removal note says the call screen, live call provider, ML voice service, and voice endpoints were deleted. Therefore the core app is text/chat/reporting ready; the call engine is a high-value accessibility differentiator that needs final screen and provider wiring before being claimed as fully demo-ready.

Security, Scale, and Reliability

The backend uses environment variables for production secrets, secure cookie settings when `DEBUG=False`, JWT rotation, API throttling, Cloudinary for media storage, and deployment health checks. It also normalizes Supabase database URLs to avoid direct-host connectivity failures on Vercel. At 10x usage, the current indexes on complaint owner, status, type, department, city/state, and complaint number support the most common list and tracking queries. The main scaling risks are AI call cost, media upload volume, cache/session growth, duplicate-check scans per active category, and dashboard aggregation load.

The largest immediate security risk is not architectural: hardcoded provider keys were found in Flutter source and historical documentation. These values must be removed, rotated, and replaced with server-side or runtime configuration before public submission. The report and deck intentionally do not print the secret values.

Impact, Demo Strategy, and Next Level

JanHelp can win because it solves a real civic operations problem end to end: citizen intake, proof validation, duplicate reduction, department routing, status tracking, feedback, reopen flow, and admin visibility. The strongest 5–7 minute demo is one complete lifecycle: a citizen reports a pothole or water

leak, uploads proof, the backend validates evidence, duplicate detection runs, the nearest department is assigned, the dashboard shows operational visibility, and the citizen tracks the case until closure.

The pitch should be honest and sharp. The core complaint platform is implemented across backend and mobile code; the production API base is centralized at `janhelps.in`; Vercel deployment support exists; and the voice/call module is a credible next-generation accessibility feature. Before final submission, the team should rotate exposed keys, remove dead imports that break `flutter analyze`, restore or hide the call entry point depending on demo readiness, run Django checks in an isolated requirements environment, seed demo categories/departments, and convert the prepared Markdown deck into no more than 15 PPT/PPTX slides. The final compiled report should be exported as `JanHelp-JanHelpSmartCity.pdf`.